CaC-ConfigMgr - Project Status & Next Steps

Date: 2026-02-27
Status: 🚧 Phase 2 In Progress - Director Integration
Branch: phase2
Commit: a976647 (Phase 1 merged to main) **Tests**: 40 unit tests passing ✅

📊 Current State

✅ Completed (Design Phase)

Area	Deliverable	Status
Vision	00-VISION.md	✅ 45 lines - Target audiences, promised values
Architecture	01-ARCHITECTURE-LOGPOINT.md	✅ 103 lines - Node types, config mapping
Inventory	10-INVENTORY-FLEET.md	✅ 328 lines - Tag-based fleet
Templates	20-TEMPLATE-HIERARCHY.md	✅ 2,072 lines - **Core spec**
Processing	30-PROCESSING-POLICIES.md	✅ 241 lines - Glue resource PP
CLI	40-CLI-WORKFLOW.md	✅ 613 lines - Complete workflow
Validation	50-VALIDATION-SPEC.md	✅ 936 lines - Full validation spec
Roadmap	99-ROADMAP.md	✅ 94 lines - Decisions tracker
ADRs	ADRS.md	✅ 7 ADRs - Architecture decisions
Coding Standards	docs/CODING-STANDARDS.md	✅ 175 lines
PDFs	specs/*.pdf	✅ 5 PDFs for review
Cleanup Plan	CLEANUP-MIGRATION.md	✅ Migration strategy

Total Specs: ~3,500 lines of documentation

🗂 Project Structure (Post-Cleanup)

```
g6-cac/
├── README.md                    # ✅ Project overview
├── ADRS.md                      # ✅ 7 Architecture decisions
├── CLEANUP-MIGRATION.md         # ✅ Migration plan
├── AGENTS.md                    # ✅ Project background
├── pyproject.toml               # ✅ Package config (cac-configmgr)
├── .gitignore                   # ✅ Standard Python
```

```
│
├── specs/                          # ✅ Specifications (source of truth)
│   ├── 00-VISION.md
│   ├── 01-ARCHITECTURE-LOGPOINT.md
│   ├── 10-INVENTORY-FLEET.md
│   ├── 20-TEMPLATE-HIERARCHY.md      # Main spec
│   ├── 30-PROCESSING-POLICIES.md
│   ├── 40-CLI-WORKFLOW.md
│   ├── 99-ROADMAP.md
│   └── *.pdf                         # PDF versions for review
│
├── docs/
│   └── CODING-STANDARDS.md         # ✅ Python standards
│
├── examples/simple/                # 🧹 Needs update to match specs
│   ├── 01-repos.yaml
│   ├── 02-routing.yaml
│   ├── 03-normalization.yaml
│   └── 04-processing.yaml
│
├── src/
│   └── cac_configmgr/              # ✅ Implementation complete
│       ├── __init__.py
│       ├── models/                 # ✅ Pydantic models (v2)
│       ├── core/                   # ✅ Resolution + validation
│       ├── providers/              # 🚧 API connectors (TODO)
│       └── cli/                    # ✅ Validate command
│
└── tmp/                            # Temporary files (gitignored)
```

🎯 Key Concepts Validated

1. Template Hierarchy

- **4 Levels**: LogPoint Golden → MSSP → Profile → Instance
- **Two Inheritance Types**: Cross-level (vertical) + Intra-level (horizontal)
- **Template IDs**: `_id` for list element matching and merge
- **List Ordering**: `_after`, `_before`, `_position`, `_first`, `_last`

2. Fleet Inventory

- **Tag-based**: Everything is tagged
- **Relationships**: `cluster`, `sh-for`, `env`
- **No hardcoded refs**: Flexible, selector-friendly

3. Processing Policies

- **Simple Glue**: RP + NP + EP references
- **No Orchestration**: Just links 3 policies together

4. CLI Workflow

- **Commands**: validate → plan → apply → drift → backup
- **Idempotent**: Safe to run multiple times
- **CI/CD Ready**: JSON output, exit codes, auto-approve

5. Multi-API Support (ADR-007)

- **Director API**: MSSP multi-tenant (MVP)
- **Direct API**: SIEM local (future)
- **Versioning**: `apiVersion`, SemVer for templates
- **Extensible**: Other products (SOAR, NDR)

🚧 Implementation Gaps

Critical (Blocking MVP)

Component	Status	Priority	Effort
Pydantic Models	✅ Implemented	P0	Medium
Template Resolution	✅ Implemented	P0	High
API Validation	✅ Implemented	P0	Medium
Validate Command	✅ Implemented	P0	Medium
Director Provider	✅ Implemented	P0	High

Important (MVP Complete)

Component	Status	Priority	Effort
Plan Command	⨯ Not started	P1	Medium
Apply Command	⨯ Not started	P1	High
Drift Command	⨯ Not started	P1	Medium
Examples Update	🧹 Needs update	P1	Low

Future (Post-MVP)

Component	Status	Priority
Direct Provider	⨯ Not started	P2
Alert Rules Spec	⨯ Not started	P2
Devices Spec	⨯ Not started	P2
GUI/TUI	⨯ Not started	P3

📋 Next Steps Plan

Phase 1: Foundation ✅ COMPLETE

Goal: Core models and validation working

Status: All components implemented and tested

1. ✅ **Pydantic Models** (`src/cac_configmgr/models/`)
   - fleet.py, template.py, repos.py, routing.py, normalization.py, processing.py, enrichment.py
   - All models from 20-TEMPLATE-HIERARCHY.md
   - Validation rules (name patterns, required fields)
   - API-compliant serialization (aliases, field names)
2. ✅ **Core Resolution** (`src/cac_configmgr/core/`)
   - resolver.py: Build inheritance chain (6 levels)
   - merger.py: Deep merge with _id matching
   - interpolator.py: Variable substitution
   - validator.py: Cross-resource consistency
   - api_validator.py: API Director compliance
   - logpoint_dependencies.py: Deployment order
3. ✅ **Validate Command** (`src/cac_configmgr/cli/main.py`)
   - 4-level validation: Syntax → References → API Compliance → Dependencies
   - Options: --fleet, --topology, --api-compliance, --offline, --verbose, --json
   - Exit codes: 0=OK, 1=warnings, 2=errors
   - Demo configs: Bank A (27 resources), Bank B (22 resources) - All pass

Deliverable: `cac-configmgr validate ./demo-configs/` works ✅

Phase 2: Director Integration 🚧 IN PROGRESS

Timeline: Week 3-4
Goal: Connect to Director API, plan/apply working
Plan: See PHASE2-PLAN.md for detailed implementation plan

Current Status: Week 1 ✅ - Provider Foundation Complete

1. **Director Provider** (Priority: P0)

```
src/cac_configmgr/providers/
├── base.py           # Abstract Provider class
└── director.py       # DirectorProvider implementation
```

   - Authentication (token, pool)
   - API client (httpx)

    - Resource mapping (CaC → Director API)
    - Async operations support

2. **Plan Command** (Priority: P1)
    - Load declared state (YAML)
    - Fetch actual state (Director API)
    - Calculate diff
    - Output: CREATE/UPDATE/DELETE table

3. **Apply Command** (Priority: P1)
    - Execute plan
    - Handle async operations (polling)
    - Error handling & rollback
    - Progress reporting

Deliverable: `plan` and `apply` working with Director

Phase 3: Polish & Drift (Week 5-6)

Goal: Production-ready CLI

1. **Drift Command** (Priority: P1)
    - Compare declared vs actual
    - Detect external changes
    - Reconcile command

2. **Backup Command** (Priority: P1)
    - Export current configuration
    - YAML format

3. **Examples Update** (Priority: P1)
    - Update `examples/simple/` to match specs
    - Add `_id` fields
    - Add complete working examples

4. **Documentation** (Priority: P1)
    - API docs (docstrings)
    - User guide
    - Migration guide from DirSync

Deliverable: Full CLI workflow operational

Phase 4: Testing & Hardening (Week 7-8)

Goal: Stable, tested, ready for pilot

Current Status: 40 unit tests already passing ✅

1. **Unit Tests** (Priority: P0) - 🚧 In Progress
   - ✅ All models (Fleet, Template, Resources)
   - ✅ Resolution algorithm
   - ✅ Merge scenarios
   - 🚧 Edge cases and error handling
2. **Integration Tests** (Priority: P0)
   - Against test Director instance
   - Full workflow tests
3. **Error Handling** (Priority: P1)
   - Edge cases
   - Network failures
   - API errors
4. **Performance** (Priority: P2)
   - Caching
   - Parallel operations

Deliverable: Pilot-ready with test coverage

🎒 DirSync Migration (Parallel Track)

Not blocking MVP, but prepare:

1. **Audit Phase** (Week 1-2)
   - Document all DirSync configurations
   - Map to CaC concepts
   - Identify gaps
2. **Migration Scripts** (Week 3-4)
   - DirSync YAML → CaC YAML converter
   - Validation tools
3. **Pilot Migration** (Week 5-8)
   - One simple client
   - Compare results
   - Validate parity

⚠ Risks & Mitigations

Risk	Impact	Mitigation
Director API changes	High	Abstract provider layer (ADR-007)
Complex merge bugs	Medium	Extensive unit tests
Performance with large configs	Medium	Caching, lazy loading
User adoption (DirSync users)	Medium	Migration guide, training

---

🎯 Success Criteria

MVP Definition (8 weeks)

- ☐ All P0 items complete
- ☐ Validate, Plan, Apply, Drift commands working
- ☐ Director API integration
- ☐ Test coverage > 80%
- ☐ Documentation complete
- ☐ One pilot client migrated from DirSync

Ready for Production (12 weeks)

- ☐ All P1 items complete
- ☐ Multiple pilot clients
- ☐ Performance validated (100+ SIEMs)
- ☐ User feedback incorporated

🚀 Immediate Actions (Next)

1. **Start Phase 2: Director Integration**
   - Implement Director Provider (`providers/director.py`)
   - Create base Provider abstract class
   - Implement name-to-ID resolution
2. **Implement Plan Command**
   - Load declared state from YAML
   - Fetch actual state from Director API
   - Calculate diff (CREATE/UPDATE/DELETE)
3. **Implement Apply Command**
   - Execute plan with proper ordering
   - Handle async operations and polling
   - Name-to-ID translation during apply

📞 Questions Resolved ✅

All major questions answered during Phase 1:

1. ✅ **State Management**: Stateless (API-only), no local state file
2. ✅ **Processing Policy**: NP/EP optional, default to "None" in API
3. ✅ **Intra-level depth**: Allow chains, 6 levels tested (Bank A)
4. ✅ **DirSync Priority**: New deployments first, migration later

New Questions for Phase 2:

1. **Apply Strategy**: Create all then update, or sequential per resource type?
2. **Error Handling**: Stop on first error or continue with rollback?
3. **Async Timeout**: How long to poll for async operations?

Status: 🔘 **Ready to start implementation**

Next immediate action: Choose first component to implement (recommendation: Pydantic models)